CAPÍTULO 5. GIT DISTRIBUIDO

Introducción

Git es un sistema de control de versiones distribuido, lo que significa que cada usuario que clona un repositorio obtiene una copia local que contiene los archivos y el historial de cambios del proyecto. Este funcionamiento permite que diferentes personas trabajen de manera independiente y compartan sus modificaciones cuando sea necesario.

A diferencia de los sistemas centralizados, en los que los usuarios dependen de un servidor principal para realizar numerosas operaciones, Git permite trabajar localmente, registrar cambios, consultar el historial y crear ramas sin necesidad de mantener una conexión permanente a internet.

En este capítulo se explican los conceptos fundamentales del trabajo distribuido, los flujos de trabajo más utilizados, la colaboración mediante repositorios remotos y algunas prácticas para contribuir a proyectos compartidos.

1. ¿Qué significa trabajar de forma distribuida?

Trabajar de forma distribuida significa que cada participante cuenta con una copia local del repositorio y puede realizar modificaciones sin intervenir directamente en la copia de los demás usuarios.

Por ejemplo, en un equipo de trabajo, una persona puede desarrollar una funcionalidad mientras otra corrige errores en una rama diferente. Posteriormente, cada participante puede compartir sus cambios con el repositorio remoto para que el equipo los revise e integre.

Este modelo ofrece varias ventajas:

- Permite trabajar sin conexión permanente a internet.
- Facilita la creación y administración de ramas.
- Mantiene un historial local de las confirmaciones.
- Favorece la colaboración entre diferentes personas.
- Reduce la dependencia de un único servidor para las operaciones locales.

Sin embargo, para compartir los cambios y mantener actualizado el trabajo colectivo, es necesario sincronizar las copias locales con los repositorios remotos.

2. Flujos de trabajo en Git

Un flujo de trabajo define la manera en que los integrantes de un proyecto colaboran, intercambian cambios y mantienen organizado el desarrollo.

Entre los modelos más utilizados se encuentran los siguientes.

- **Flujo de trabajo centralizado**

En este modelo existe un repositorio remoto principal al que los colaboradores envían sus confirmaciones. Cada participante trabaja en una copia local y sincroniza sus modificaciones con el servidor.

Los comandos más habituales son:

```
git pull origin main

git add .

git commit -m "Describe los cambios realizados"

git push origin main
```

El primer comando obtiene e integra los cambios de la rama remota main. Los siguientes preparan los archivos, registran una confirmación y envían los cambios al servidor.

Este flujo es sencillo para equipos pequeños, pero requiere coordinación para evitar conflictos cuando varias personas modifican los mismos archivos.

- **Flujo de trabajo con ramas de funcionalidad**

En este modelo, cada nueva funcionalidad o corrección se desarrolla en una rama independiente. Una vez terminada, se revisa y se integra en la rama principal.

Por ejemplo:

```
git switch -c nueva-funcionalidad
```

Después de realizar los cambios, se registran:

```
git add .

git commit -m "Agrega nueva funcionalidad"
```

La rama puede enviarse al repositorio remoto mediante:

```
git push -u origin nueva-funcionalidad
```

Posteriormente, los cambios pueden revisarse mediante una solicitud de incorporación de cambios (*pull request*) en plataformas como GitHub y, después de su aprobación, fusionarse en la rama principal.

Este método facilita la revisión del código y evita que el trabajo que todavía está en desarrollo afecte directamente la versión principal.

- **Flujo de trabajo basado en bifurcaciones**

Este modelo, conocido como *forking workflow*, se utiliza especialmente en proyectos de código abierto. Cada colaborador crea una bifurcación o *fork* del repositorio original dentro de su propia cuenta.

El proceso general consiste en:

1. Crear una bifurcación del repositorio original en GitHub.
2. Clonar la bifurcación en el computador.
3. Crear una rama para desarrollar los cambios.
4. Registrar las modificaciones mediante confirmaciones.
5. Enviar la rama a la bifurcación personal.
6. Abrir una solicitud de incorporación de cambios para proponer las modificaciones al proyecto original.

Este modelo permite que las personas contribuyan a proyectos incluso cuando no tienen permisos de escritura sobre el repositorio original.

3. Colaborar mediante repositorios remotos

Los repositorios remotos permiten intercambiar cambios entre los integrantes de un equipo. Para colaborar correctamente, es importante conocer los comandos de descarga, actualización y envío.

- **Obtener los cambios**

Para descargar la información más reciente de un repositorio remoto sin integrar automáticamente sus modificaciones en la rama actual, se utiliza:

git fetch origin

Este comando actualiza la información local sobre las ramas remotas y permite revisar las nuevas confirmaciones antes de incorporarlas al trabajo propio.

- **Integrar los cambios**

Para obtener cambios de una rama remota e integrarlos en la rama local actual, se puede utilizar:

git pull origin main

Este comando obtiene los cambios de la rama remota main y los integra de acuerdo con la configuración de Git.

Antes de utilizarlo, es recomendable comprobar el estado del repositorio y asegurarse de que los cambios locales estén guardados o registrados.

- **Enviar las modificaciones**

Una vez que los cambios están registrados localmente, se pueden enviar al repositorio remoto mediante:

```
git push origin main
```

Este comando intenta actualizar la rama remota main con las confirmaciones locales. Para ejecutarlo correctamente, el usuario debe tener los permisos necesarios y la rama local debe poder integrarse con la rama remota.

En caso de que existan cambios remotos que no estén presentes localmente, Git puede rechazar el envío. En ese caso, es necesario obtener e integrar los cambios correspondientes antes de volver a intentarlo.

4. **Sincronizar una bifurcación con el repositorio original**

Cuando se trabaja mediante una bifurcación, el repositorio personal puede quedar desactualizado respecto del proyecto original. Para mantenerlo actualizado, se puede agregar el repositorio original como un remoto adicional.

Por ejemplo:

```
git remote add upstream https://github.com/organizacion/proyecto.git
```

En este ejemplo, upstream identifica el repositorio original del que se creó la bifurcación. La dirección debe reemplazarse por la del proyecto real.

Para verificar los repositorios remotos configurados, se utiliza:

```
git remote -v
```

Posteriormente, se pueden descargar los cambios del repositorio original:

```
git fetch upstream
```

Si se desea integrar los cambios de su rama principal en la rama local main, se puede ejecutar:

```
git switch main
```

```
git merge upstream/main
```

Este procedimiento permite actualizar la copia local con los cambios del proyecto original. Si existen modificaciones incompatibles, Git puede solicitar la resolución de conflictos antes de completar la integración.

5. **Solicitudes de incorporación de cambios en GitHub**

Las solicitudes de incorporación de cambios, conocidas como *pull requests*, son una herramienta de colaboración que permite proponer modificaciones para que otras personas las revisen antes de integrarlas en una rama.

En GitHub, este proceso generalmente incluye los siguientes pasos:

1. Crear una rama para desarrollar una funcionalidad o corregir un error.
2. Realizar las modificaciones y registrarlas mediante confirmaciones.
3. Enviar la rama al repositorio remoto.
4. Abrir una solicitud de incorporación de cambios desde la interfaz de GitHub.
5. Describir el propósito de las modificaciones.
6. Revisar los comentarios y realizar los ajustes necesarios.
7. Fusionar los cambios cuando se cumplan las condiciones del proyecto.

Las solicitudes de incorporación de cambios facilitan la revisión del trabajo, la discusión de propuestas y la detección de errores antes de modificar la rama principal.

Aunque el nombre incluye la palabra *pull*, una solicitud de incorporación de cambios no equivale al comando git pull: se trata de una herramienta de colaboración ofrecida por plataformas como GitHub.

6. Buenas prácticas para el trabajo colaborativo

Para mantener un repositorio organizado y facilitar la colaboración, se recomienda aplicar las siguientes prácticas:

- **Realizar confirmaciones claras:** utilizar mensajes que describan brevemente el cambio realizado.
- **Trabajar con ramas:** desarrollar funcionalidades y correcciones en ramas separadas cuando el proyecto lo requiera.
- **Sincronizar con frecuencia:** obtener los cambios remotos antes de integrar nuevas modificaciones propias.
- **Revisar el estado del repositorio:** ejecutar git status para identificar archivos modificados, preparados o sin seguimiento.
- **Evitar confirmaciones innecesarias:** revisar los archivos antes de ejecutar git add.
- **Resolver conflictos cuidadosamente:** comparar las modificaciones y comprobar que el resultado final sea correcto.
- **Proteger la información privada:** no subir contraseñas, tokens, claves privadas ni otros datos confidenciales al repositorio.
- **Revisar los cambios antes de fusionarlos:** comprobar que las modificaciones cumplan los objetivos del proyecto.

Estas prácticas permiten reducir errores, mejorar la trazabilidad de las modificaciones y mantener una colaboración más eficiente.

7. Ejercicio práctico

El siguiente ejercicio permite practicar la creación de una rama, el registro de cambios y el envío de una modificación a GitHub.

Paso 1. Consultar el estado del repositorio:

```
git status
```

Paso 2. Crear una rama para una nueva modificación:

```
git switch -c practica-colaboracion
```

Paso 3. Crear un archivo de prueba:

```
echo "Ejercicio de trabajo distribuido con Git" > colaboracion.txt
```

Paso 4. Agregar el archivo al área de preparación:

```
git add colaboracion.txt
```

Paso 5. Registrar la modificación:

```
git commit -m "Agrega ejercicio de colaboracion"
```

Paso 6. Enviar la rama al repositorio remoto:

```
git push -u origin practica-colaboracion
```

Este paso requiere que el repositorio remoto esté configurado y que el usuario tenga permisos de escritura.

Paso 7. Ingresar al repositorio en GitHub y verificar que la rama practica-colaboracion y el archivo colaboracion.txt estén disponibles.

Paso 8. Si el proyecto lo permite, crear una solicitud de incorporación de cambios desde la rama de práctica hacia la rama principal.

Al terminar, se habrá practicado un flujo básico de colaboración: crear una rama, registrar una modificación, enviarla al servidor y proponer su incorporación al proyecto.

Conclusión

El modelo distribuido de Git permite que cada colaborador trabaje con una copia local completa del repositorio y comparta sus modificaciones mediante servidores remotos. Los diferentes flujos de trabajo, como el centralizado, el basado en ramas y el de bifurcaciones, se adaptan a las necesidades de distintos equipos y proyectos.

Asimismo, el uso de comandos como git fetch, git pull, git push y git merge, junto con herramientas de colaboración como las solicitudes de incorporación de cambios de GitHub, permite mantener un proceso de desarrollo organizado y controlado. Aplicar buenas prácticas de sincronización, revisión y seguridad ayuda a prevenir errores y facilita el trabajo conjunto.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Capítulo 5: Git distribuido. Disponible en: https://git-scm.com/book/es/v2/Git-distribuido